

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 04 · LECTURE NOTES

Sorting and Selection: Limits and Workarounds

A proven lower bound, and the legitimate ways around it.

MODULE OUTCOMES

Prove the $\Omega(n \log n)$ comparison-sorting lower bound.

Explain exactly which assumption the bound depends on.

Apply counting, radix, and bucket sort, and state their requirements.

Explain stability and when it matters.

Describe what production sorts actually do and why.

1 The comparison lower bound

Most of this course is about finding better algorithms. This section is about proving that no better algorithm exists, which is a rarer and in some ways more satisfying kind of result.

1.1 The decision tree model

Model any comparison-based sorting algorithm as a binary tree. Each internal node is a comparison 'is $a_i \leq a_j$?'; the two children correspond to the two answers. Execution is a root-to-leaf path, and each leaf must be labelled with the permutation the algorithm outputs when it arrives there.

- 1 To sort correctly, the algorithm must be able to output any of the $n!$ possible permutations. So the tree has at least $n!$ leaves.
- 2 A binary tree of height h has at most 2^h leaves. Hence $2^h \geq n!$, so $h \geq \log_2(n!)$.
- 3 By Stirling's approximation, $\log_2(n!) = n \log_2 n - n \log_2 e + O(\log n) = \Theta(n \log n)$.
- 4 The height is the worst-case number of comparisons. Therefore every comparison sort makes $\Omega(n \log n)$ comparisons in the worst case.

WHY THIS IS A STRONG RESULT

This is not a statement about mergesort or quicksort. It is a statement about *every comparison-based sorting algorithm that will ever be written*, including ones nobody has thought of.

Results of this kind are uncommon. Most of the time we can only say that nobody currently knows how to do better — the P versus NP question (Module 12) is exactly that situation. Here we genuinely know.

1.2 The assumption, stated plainly

The proof assumes the algorithm learns about the input *only* through comparisons. Each comparison yields one bit, there are $\log_2(n!)$ bits of information to acquire, so you need that many comparisons. Stated that way the result is almost obvious, which is a good sign.

And it immediately tells you how to beat it: extract more than one bit per operation. If a key can be used as an array index, that one operation distinguishes among k possibilities rather than two. The decision tree is then not a model of your algorithm and the bound simply does not apply to it.

2 Sorting in linear time

2.1 Counting sort

For integer keys in $[0, k)$: count occurrences of each value, convert the counts to prefix sums to get output positions, then place elements. $\Theta(n + k)$ time and $\Theta(n + k)$ space.

```
def counting_sort(a, k):
    count = [0] * k
    for x in a:
        count[x] += 1          # histogram

    total = 0                  # prefix sums -> start positions
    for i in range(k):
        count[i], total = total, total + count[i]

    out = [None] * len(a)
    for x in a:
        out[count[x]] = x      # forward pass over the input keeps
        count[x] += 1          # equal keys in original order
                                # -> stable
    return out
```

Usable only when k is comparable to n . Sorting 32-bit integers directly would need an array of four billion counters, so counting sort is not used alone for wide keys — it is used as the inner loop of radix sort.

2.2 Radix sort

Treat each key as d digits in base k . Sort by the least significant digit, then the next, and so on to the most significant, using a stable sort at each pass.

STABILITY IS LOAD-BEARING

After the pass on digit i , two elements agreeing on digit i must remain in the relative order that the earlier passes gave them — that is precisely what encodes the information from the lower-order digits.

Use an unstable sort for the inner pass and radix sort does not merely slow down; it produces wrong output. This is the clearest example in the course of stability being a correctness property rather than a convenience.

$\Theta(d(n + k))$. For 32-bit integers split into four 8-bit digits, that is four passes with $k = 256$ — linear in n with a small constant, and genuinely faster than comparison sorting for large arrays of fixed-width integers.

2.3 Bucket sort

Distribute elements into buckets by value range, sort each bucket, concatenate. Linear in expectation *if* the keys are roughly uniform over the range. Under a skewed distribution everything lands in one bucket and the cost degrades to that of the inner sort, which is the honest reason bucket sort is rarely used outside settings where the distribution is known.

3 Stability

A sort is **stable** if elements comparing equal keep their original relative order.

Stable	Unstable
Mergesort, Timsort, counting sort, insertion sort, bubble sort	Quicksort, heapsort, introsort, selection sort

Three reasons it matters:

- 1 **Radix sort requires it** for correctness, as above.
- 2 **Multi-key sorting by repeated passes.** Sort by surname, then stably by department, and you get a listing grouped by department and alphabetical within each. Without stability the second pass destroys the first.
- 3 **User-visible ordering.** Re-sorting a displayed table by a new column should not scramble rows that tie. Java's `Arrays.sort` uses Timsort for objects specifically because API users expect this.

Stability is not free — it typically costs auxiliary memory — which is why C++ offers both `std::sort` and `std::stable_sort` and makes you choose.

4 What production sorts actually do

Implementation	Algorithm	Why
C++ <code>std::sort</code>	Introsort	Quicksort for speed; switches to heapsort when recursion depth exceeds $\sim 2 \log n$, guaranteeing $O(n \log n)$ worst case; insertion sort below ~ 16 elements where its constant wins.
Python <code>sorted()</code>	Timsort	Detects existing ascending or descending runs and merges them. $\Theta(n)$ on already-sorted input, stable, and tuned for the partly ordered data that occurs constantly in practice.
Java <code>Arrays.sort</code>	Dual-pivot quicksort / Timsort	Dual-pivot quicksort for primitives, where stability is meaningless; Timsort for objects, where callers expect stability.
Rust <code>sort_unstable</code>	Pattern-defeating quicksort	Detects patterns that would trigger quicksort's bad cases and adapts, retaining quicksort's speed without the vulnerability.

THE LESSON

Not one production sort is a textbook algorithm. Every one is a hybrid that switches strategy based on input size, detected structure, and recursion depth.

Textbook algorithms are *components*. The engineering is in knowing which component applies when — and that judgement comes from exactly the analysis this course teaches.

Timsort in particular is fast because real data is frequently partly sorted, a fact worst-case analysis is structurally unable to see. Worst-case bounds tell you what cannot go wrong; they do not tell you what usually happens.

5 Other lower bounds

Problem	Bound	Argument
Search a sorted array	$\Omega(\log n)$	n possible answers, one bit per comparison.
Find the maximum	Exactly $n - 1$	Every element except the maximum must lose at least one comparison, and each comparison produces one loser.
Find max and min together	$\lceil 3n/2 \rceil - 2$	Pair elements first, then compare winners and losers separately. Beats the naive $2n - 3$.
Element uniqueness	$\Omega(n \log n)$	In the comparison model. Hashing beats it, by changing the model.
Sorting	$\Omega(n \log n)$	The decision tree argument of §1.

All share a shape: count how many distinct outputs are possible, bound how much information one permitted operation can yield, and divide. Recognising that shape lets you derive bounds for new problems rather than memorising them.

The practical value is knowing when to stop. If you have a comparison sort running at $n \log n$, no amount of cleverness within that model will improve the asymptotics, and your remaining options are constant factors or a different model. That is a genuinely useful thing to know on a deadline.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.006 — Sorting lower bound, Counting and Radix Sort	The decision tree argument presented carefully, with the linear-time sorts as the follow-on.
Jeff Erickson, Algorithms — Sorting chapter	Good on what the lower bound does and does not constrain.
Tim Peters — Timsort description (CPython source)	The design document for Timsort, written by its author. An unusually clear piece of engineering writing — read it.
Orson Peters — Pattern-defeating quicksort	How Rust's unstable sort resists adversarial patterns. Short and readable.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write out the decision tree for sorting three elements. Confirm it has 6 leaves and height 3, and check this against the $\log_2(3!)$ bound.
- 2 Implement counting sort and radix sort. Measure radix sort against `std::sort` (or `sorted()`) on 10^7 32-bit integers and report the ratio.
- 3 Break stability in your counting sort by iterating the final pass backwards without adjusting the indices, then run radix sort with it. Capture the wrong output and explain exactly which information was lost.
- 4 Implement bucket sort and measure it on uniform input and on exponentially distributed input. Show the degradation and explain it.
- 5 Implement the pairing algorithm that finds max and min in $\lceil 3n/2 \rceil - 2$ comparisons. Verify the count empirically against the naive $2n - 3$.
- 6 Generate a nearly-sorted array (sorted, then 1% of elements swapped at random). Measure Timsort or `sorted()` against `std::sort` on it. Explain the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Reproduce the decision-tree lower bound argument, including why the tree must have $n!$ leaves.
- 2 What single assumption does the $\Omega(n \log n)$ bound rest on, and how do linear-time sorts evade it?
- 3 State the requirement and the failure mode of counting, radix, and bucket sort.
- 4 Why is stability a correctness requirement for radix sort rather than a convenience?
- 5 Give two reasons a production sort would switch algorithms partway through.
- 6 Why is Timsort fast on real data in a way that worst-case analysis cannot predict?